Task 1: Customer Churn Prediction Using a Shallow Neural Network

CN6021 — Advanced Topics in AI and Data Science

Group [Your Group Name/Number]

2026-04-12

Table of contents

1 Introduction **1**

2 Methodology **1**

2.1 Data Loading and Exploratory Data Analysis . . . 1

2.1.1 Missing Values . . . 3

2.1.2 Correlation Analysis . . . 4

2.1.3 Feature Distributions by Churn Status . . . 5

2.2 Data Preprocessing . . . 6

2.3 Feature Selection and Dimensionality Reduction . . . 8

2.3.1 Correlation-Based Removal . . . 8

2.3.2 Mutual Information Feature Selection . . . 8

2.3.3 PCA Visualisation . . . 10

2.4 Addressing Class Imbalance . . . 11

2.5 Shallow Neural Network Implementation . . . 12

2.5.1 Architecture Design . . . 12

2.6 Hyperparameter Tuning . . . 16

2.7 Training the Final Model . . . 17

3 Results **18**

3.1 Evaluation Metrics . . . 18

3.2 Interpretability Analysis . . . 21

4 Conclusions **24**

5 Appendix **25**

5.1 How to Run . . . 25

5.2 Output Files . . . 25

1 Introduction

Customer churn — the loss of clients or subscribers — is a critical challenge for subscription-based and e-commerce businesses. Predicting which customers are likely to leave allows organisations to intervene proactively, reducing attrition and improving lifetime value. This report presents a **shallow neural network built from scratch using NumPy** to predict customer churn, addressing key data challenges including high dimensionality, non-linear relationships, and class imbalance.

Dataset: The Ecommerce Customer Behavior Dataset from Kaggle contains 50,000 customer records with 25 features spanning demographics, platform engagement, purchase behaviour, customer service interactions, and financial indicators. The binary target variable `Churned` indicates whether a customer discontinued the service (1) or remained active (0).

Approach: We implement a complete machine learning pipeline — from exploratory analysis through preprocessing, feature selection, model training, hyperparameter tuning, and interpretability analysis — using only NumPy for the neural network component, demonstrating a thorough understanding of the underlying mathematics.

2 Methodology

2.1 Data Loading and Exploratory Data Analysis

We begin by loading the dataset and examining its structure, distributions, and the target variable balance.

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_auc_score,
    roc_curve,
    precision_recall_fscore_support,
    f1_score,
)
from sklearn.feature_selection import mutual_info_classif
from sklearn.decomposition import PCA

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
```

```python
df = pd.read_csv("data/ecommerce_customer_churn_dataset.csv")
print(f"Dataset shape: {df.shape}")
print(f"Churn rate: {df['Churned'].mean()*100:.1f}%")
```

```
Dataset shape: (50000, 25)
Churn rate: 28.9%
```

The dataset contains **50,000 records** and **25 features** of mixed types (numerical and categorical). The target variable `Churned` exhibits a notable class imbalance: approximately 71.1% of customers are retained while 28.9% have churned, yielding a 2.46:1 ratio.

```python
fig, ax = plt.subplots(figsize=(6, 4))
counts = df["Churned"].value_counts()
bars = ax.bar(["Retained (0)", "Churned (1)"], counts.values,
              color=["#2ecc71", "#e74c3c"], edgecolor="black")
for bar, val in zip(bars, counts.values):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 300,
            str(val), ha="center", fontweight="bold")
ax.set_title("Target Variable Distribution", fontsize=14, fontweight="bold")
ax.set_ylabel("Count")
plt.tight_layout()
plt.show()
```

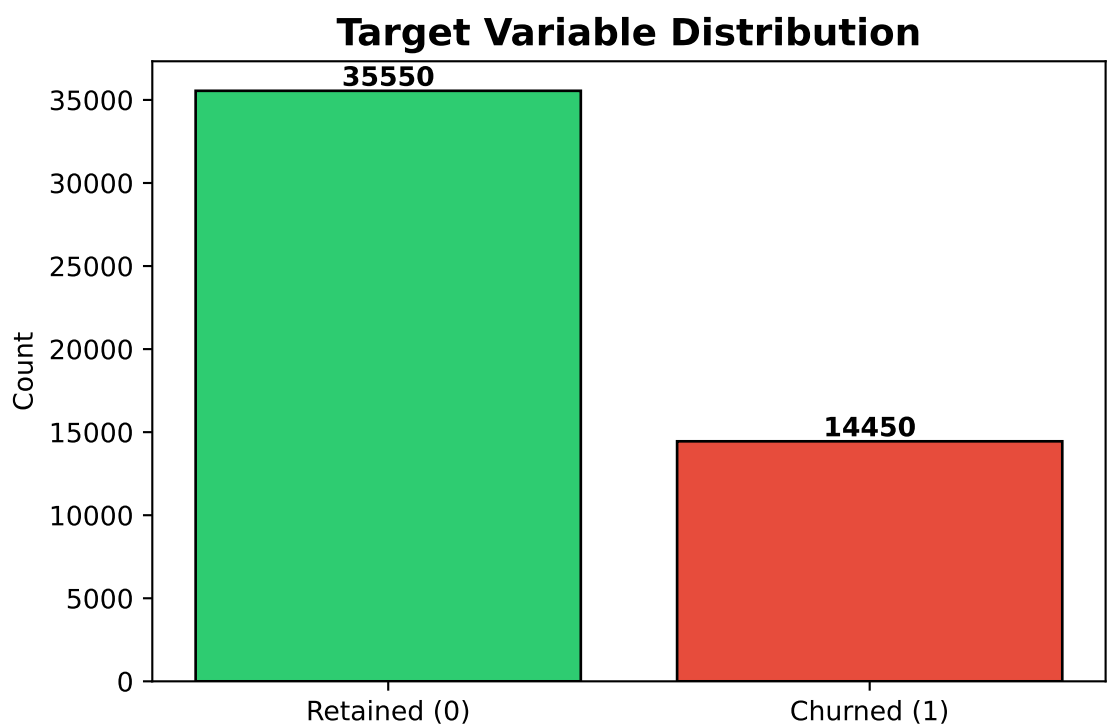

Figure 1: Target Variable Distribution

2.1.1 Missing Values

Several features contain significant missing data, with `Social_Media_Engagement_Score` having the highest at 6,000 (12%) missing values. This necessitates careful imputation during preprocessing.

```
missing = df.isnull().sum()
missing_pct = (missing / len(df) * 100).round(1)
missing_df = pd.DataFrame({"Missing Count": missing, "Percentage (%)": missing_pct})
print(missing_df[missing_df["Missing Count"] > 0].sort_values("Missing Count", ascending=Fal
```

```
                              Missing Count  Percentage (%)
Social_Media_Engagement_Score          6000            12.0
Credit_Balance                         5500            11.0
Mobile_App_Usage                       5000            10.0
Returns_Rate                           4491             9.0
Wishlist_Items                         4000             8.0
Discount_Usage_Rate                    3500             7.0
Product_Reviews_Written                3500             7.0
Session_Duration_Avg                   3399             6.8
Pages_Per_Session                      3000             6.0
Days_Since_Last_Purchase               3000             6.0
Email_Open_Rate                        2528             5.1
Payment_Method_Diversity               2500             5.0
Age                                    2495             5.0
Customer_Service_Calls                  168             0.3
```

2.1.2 Correlation Analysis

The correlation heatmap reveals the relationships between numerical features. No pairs exhibit extremely high collinearity ($|r| > 0.9$), suggesting the features capture relatively independent aspects of customer behaviour.

```
numerical_cols = df.select_dtypes(include=[np.number]).columns.tolist()
corr = df[numerical_cols].corr()
fig, ax = plt.subplots(figsize=(14, 12))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdBu_r", center=0,
            square=True, linewidths=0.5, ax=ax, annot_kws={"size": 7})
ax.set_title("Correlation Heatmap of Numerical Features", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.show()
```

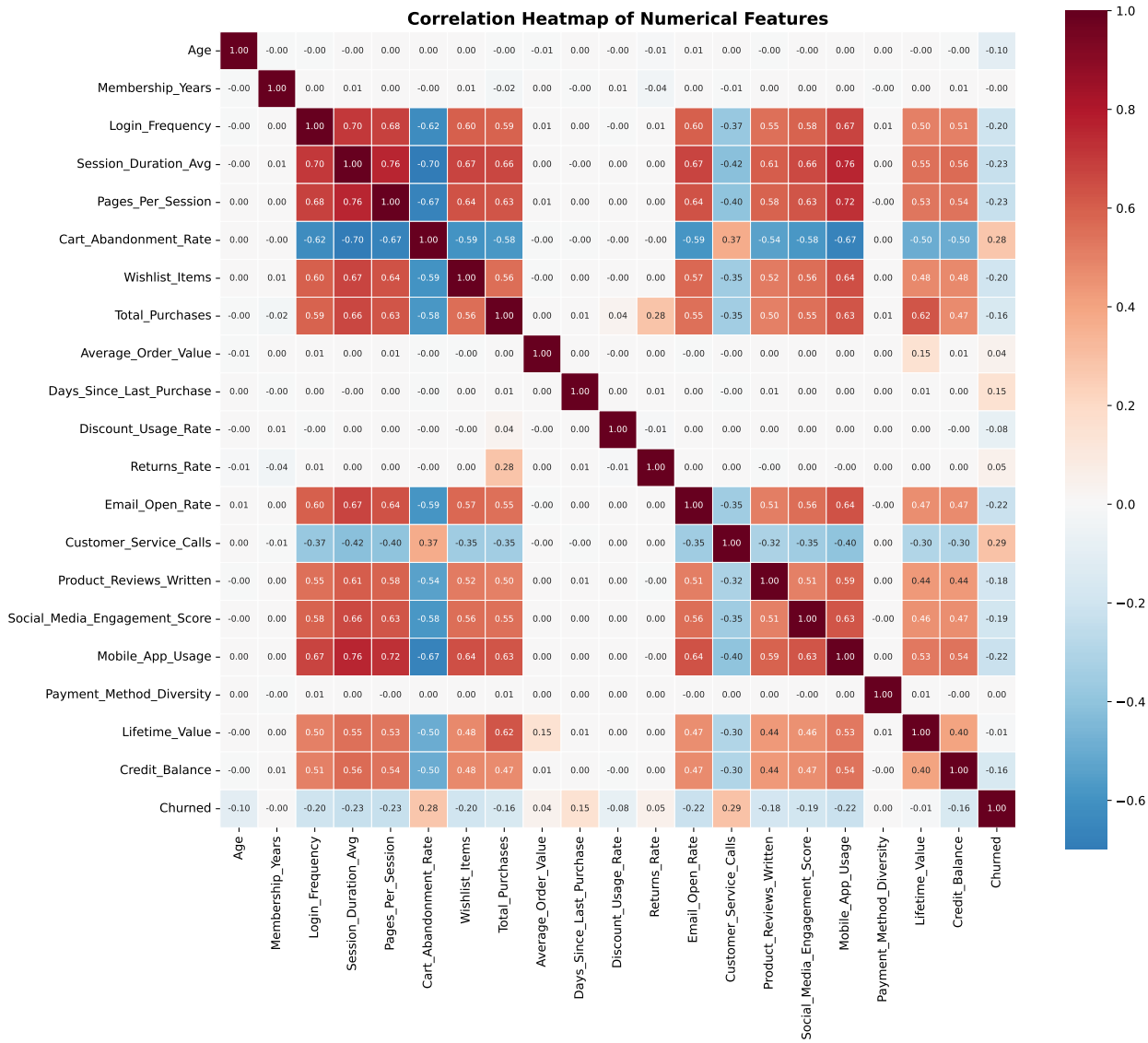

Figure 2: Correlation Heatmap of Numerical Features

2.1.3 Feature Distributions by Churn Status

Visualising feature distributions split by churn status helps identify potential predictive signals. Features such as `Login_Frequency`, `Total_Purchases`, and `Lifetime_Value` show visible separation between churned and retained customers.

```
key_features = ["Age", "Membership_Years", "Login_Frequency",
                "Total_Purchases", "Lifetime_Value", "Credit_Balance"]
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
for ax, feat in zip(axes.flat, key_features):
    for label, colour in [(0, "#2ecc71"), (1, "#e74c3c")]:
        subset = df[df["Churned"] == label][feat].dropna()
```

```
        ax.hist(subset, bins=30, alpha=0.6,
                label=f"{'Retained' if label==0 else 'Churned'}",
                color=colour, edgecolor="black", linewidth=0.5)
    ax.set_title(feat, fontsize=12, fontweight="bold")
    ax.legend(fontsize=9)
plt.suptitle("Feature Distributions by Churn Status",
             fontsize=15, fontweight="bold", y=1.01)
plt.tight_layout()
plt.show()
```

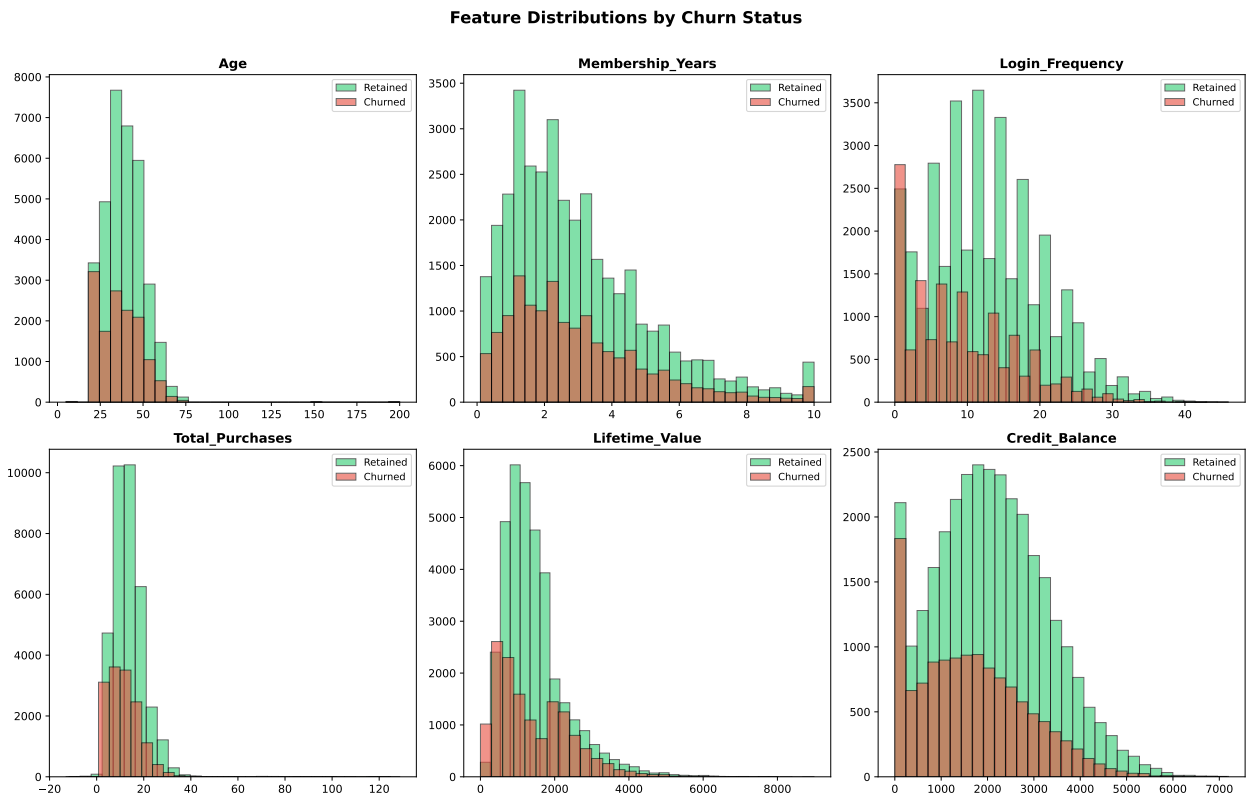

Figure 3: Feature Distributions by Churn Status

2.2 Data Preprocessing

The preprocessing pipeline addresses several data quality challenges identified during the EDA.

```
# 1. Drop 'City' - 40 unique values creates excessive dimensionality
df.drop(columns=["City"], inplace=True)

# 2. Handle outliers
df["Age"] = df["Age"].clip(upper=100)              # Age max was 200
df["Total_Purchases"] = df["Total_Purchases"].clip(lower=0)  # Remove negatives
```

```python
# 3. Impute missing values
num_cols = df.select_dtypes(include=[np.number]).columns.drop("Churned").tolist()
cat_cols = df.select_dtypes(include=["object", "string"]).columns.tolist()

for col in num_cols:
    df[col] = df[col].fillna(df[col].median())     # Median: robust to outliers

for col in cat_cols:
    df[col] = df[col].fillna(df[col].mode()[0])     # Mode: most common category

# 4. One-hot encode categorical features (drop_first to avoid dummy trap)
df = pd.get_dummies(df, columns=cat_cols, drop_first=True)

# 5. Separate features and target
y = df["Churned"].values
X = df.drop(columns=["Churned"]).values
feature_names = df.drop(columns=["Churned"]).columns.tolist()

# 6. Stratified train/test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)

# 7. Feature scaling (fit on train only to prevent data leakage)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test  = scaler.transform(X_test)

print(f"Shape after preprocessing: {X_train.shape[1]} features")
print(f"Train: {X_train.shape[0]} | Test: {X_test.shape[0]}")
print(f"Missing values remaining: {df.isnull().sum().sum()}")
```

```
Shape after preprocessing: 32 features
Train: 40000 | Test: 10000
Missing values remaining: 0
```

Key preprocessing decisions:

- **City dropped:** With 40 unique values, one-hot encoding would add excessive dimensionality without proportional predictive value.
- **Median imputation:** Chosen over mean imputation as it is robust to the outliers identified in the data (e.g., Age = 200).
- **StandardScaler:** Normalises features to zero mean and unit variance, which is essential for gradient-based optimisation — without scaling, features with larger ranges would disproportionately influence weight updates.
- **Stratified split:** Preserves the class distribution (71.1%/28.9%) in both train and test sets.

2.3 Feature Selection and Dimensionality Reduction

We apply two complementary techniques to reduce dimensions and identify the most predictive features.

2.3.1 Correlation-Based Removal

Features with pairwise correlation $|r| > 0.90$ are candidates for removal to eliminate multicollinearity, which can cause unstable gradient updates.

```
corr_matrix = np.corrcoef(X_train, rowvar=False)
n_feats = corr_matrix.shape[0]
to_drop_idx = set()
for i in range(n_feats):
    for j in range(i + 1, n_feats):
        if abs(corr_matrix[i, j]) > 0.90:
            to_drop_idx.add(j)

if to_drop_idx:
    dropped = [feature_names[i] for i in to_drop_idx]
    keep_idx = [i for i in range(n_feats) if i not in to_drop_idx]
    X_train = X_train[:, keep_idx]
    X_test  = X_test[:, keep_idx]
    feature_names = [feature_names[i] for i in keep_idx]
    print(f"Dropped {len(to_drop_idx)} features: {dropped}")
else:
    print("No features removed - no pairs exceed |r| > 0.90 threshold")
```

```
No features removed - no pairs exceed |r| > 0.90 threshold
```

2.3.2 Mutual Information Feature Selection

Mutual Information (MI) quantifies the statistical dependency between each feature and the target, capturing both linear and non-linear relationships. Features with MI 0.001 are removed as they contribute negligible predictive information.

```
mi_scores = mutual_info_classif(X_train, y_train, random_state=RANDOM_STATE)
mi_df = pd.DataFrame({"Feature": feature_names, "MI_Score": mi_scores}).sort_values(
    "MI_Score", ascending=False
)

fig, ax = plt.subplots(figsize=(10, 7))
mi_sorted = mi_df.sort_values("MI_Score", ascending=True)
ax.barh(mi_sorted["Feature"], mi_sorted["MI_Score"],
        color="#3498db", edgecolor="black", linewidth=0.5)
```

```python
ax.set_xlabel("Mutual Information Score")
ax.set_title("Feature Importance - Mutual Information with Target",
             fontsize=13, fontweight="bold")
plt.tight_layout()
plt.show()

# Select features with MI > threshold
selected_mask = mi_scores > 0.001
X_train = X_train[:, selected_mask]
X_test  = X_test[:, selected_mask]
feature_names = [f for f, keep in zip(feature_names, selected_mask) if keep]
print(f"\nRetained {sum(selected_mask)}/{len(selected_mask)} features (MI > 0.001)")
```

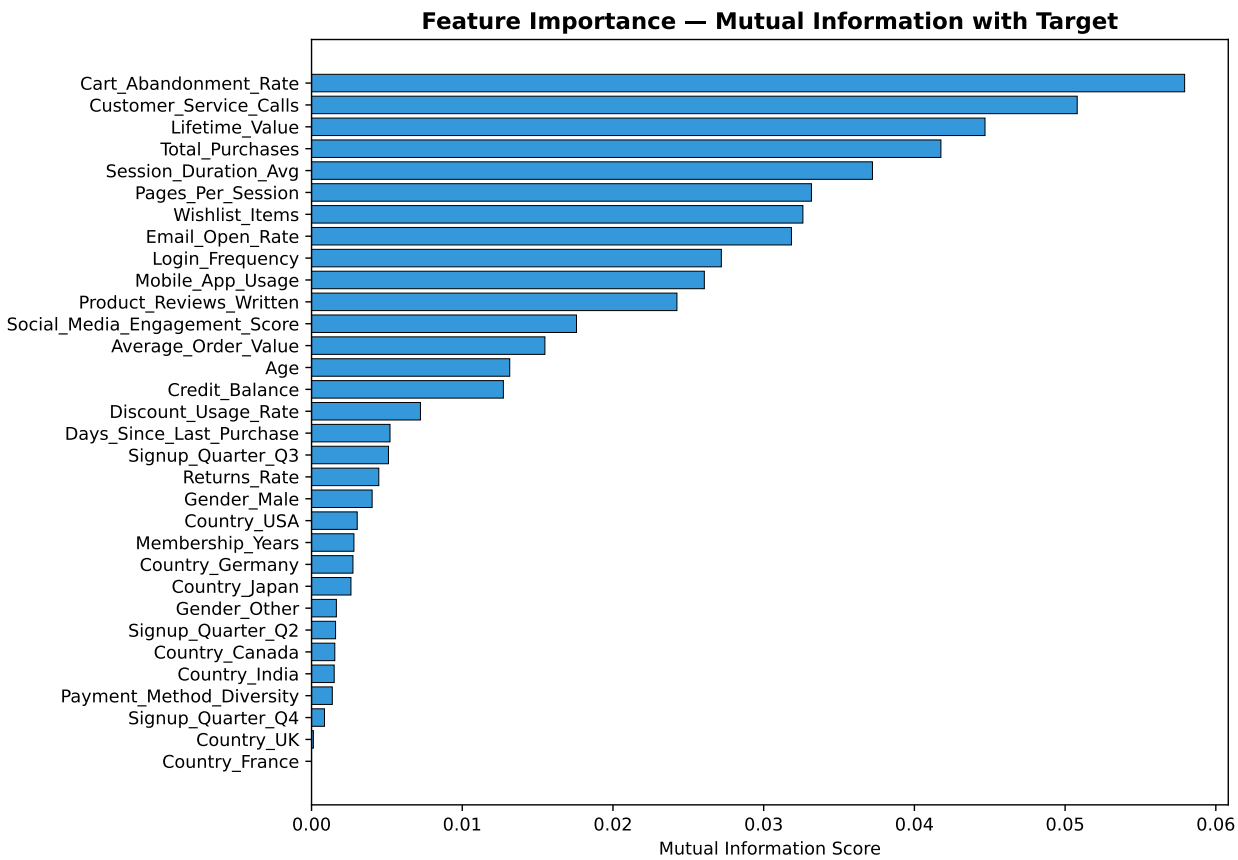

Figure 4: Mutual Information Scores — Feature Importance Ranking

```
Retained 29/32 features (MI > 0.001)
```

The top features by MI score are **Cart_Abandonment_Rate**, **Customer_Service_Calls**, and **Lifetime_Value**, suggesting that engagement behaviour and financial metrics are the strongest predictors of churn.

2.3.3 PCA Visualisation

While we do not use PCA for dimensionality reduction in the model (to preserve interpretability), we project the data onto two principal components for visualisation purposes.

```
pca = PCA(n_components=2, random_state=RANDOM_STATE)
X_pca = pca.fit_transform(X_train)
fig, ax = plt.subplots(figsize=(8, 6))
scatter = ax.scatter(X_pca[:, 0], X_pca[:, 1], c=y_train,
                     cmap="RdYlGn_r", alpha=0.3, s=8, edgecolors="none")
ax.set_xlabel(f"PC1 ({pca.explained_variance_ratio_[0]*100:.1f}% variance)")
ax.set_ylabel(f"PC2 ({pca.explained_variance_ratio_[1]*100:.1f}% variance)")
ax.set_title("PCA - 2D Projection Coloured by Churn Status",
             fontsize=13, fontweight="bold")
cbar = plt.colorbar(scatter, ax=ax)
cbar.set_ticks([0, 1])
cbar.set_ticklabels(["Retained", "Churned"])
plt.tight_layout()
plt.show()
```

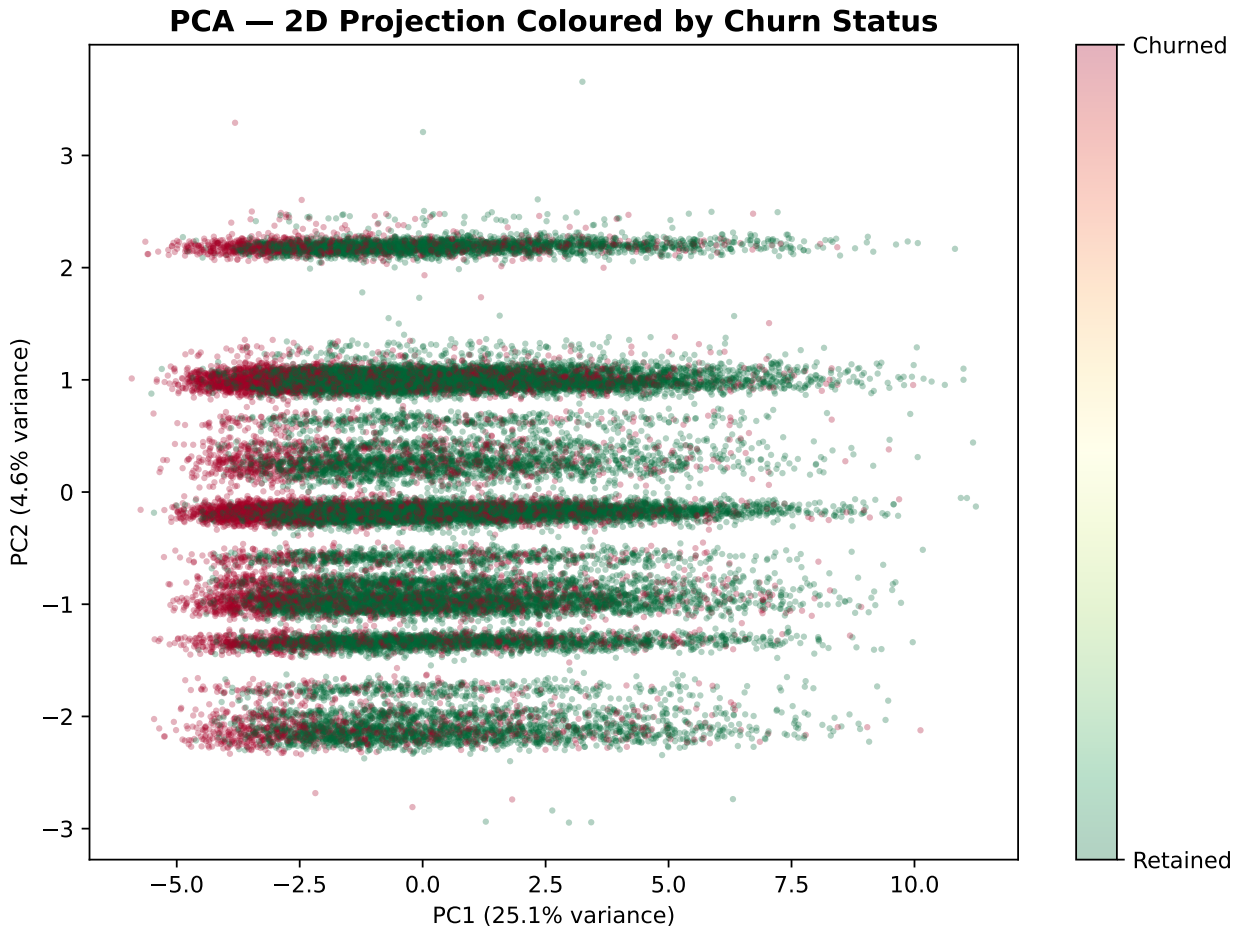

Figure 5: PCA — 2D Projection Coloured by Churn Status

The PCA scatter shows partial separation between classes, confirming that the feature space contains discriminative information, though the overlap indicates the problem is non-trivial and requires non-linear modelling.

2.4 Addressing Class Imbalance

The dataset has a 2.46:1 imbalance (retained:churned). Without correction, the model would be biased towards predicting the majority class. We address this through **weighted binary cross-entropy loss**, which assigns a higher penalty to misclassifying churned customers:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} w_i \left[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)\right]$$

where the sample weights are computed as:

$$w_{\text{churned}} = \frac{N_{\text{total}}}{2 \times N_{\text{churned}}}, \quad w_{\text{retained}} = \frac{N_{\text{total}}}{2 \times N_{\text{retained}}}$$

```python
n_total = len(y_train)
n_pos   = y_train.sum()
n_neg   = n_total - n_pos
w_pos   = n_total / (2 * n_pos)
w_neg   = n_total / (2 * n_neg)
print(f"Class weights - Retained: {w_neg:.4f} | Churned: {w_pos:.4f}")
print(f"Churned class receives {w_pos/w_neg:.2f}x the weight of retained class")
```

```
Class weights - Retained: 0.7032 | Churned: 1.7301
Churned class receives 2.46x the weight of retained class
```

This ensures the model treats both classes with equal importance during training, preventing the common pitfall of high accuracy but poor minority-class recall.

2.5 Shallow Neural Network Implementation

We implement a single hidden-layer neural network entirely in NumPy, without deep learning frameworks. This fulfils the coursework requirement for a **shallow network** while demonstrating understanding of forward propagation, backpropagation, and gradient descent from first principles.

2.5.1 Architecture Design

```
Input (29 features) → Hidden Layer (32 units, ReLU) → Output (1 unit, Sigmoid)
```

```python
# ---- Activation Functions ----
def relu(z):
    """ReLU: introduces non-linearity, avoids vanishing gradient problem."""
    return np.maximum(0, z)

def relu_derivative(z):
    """Derivative of ReLU for backpropagation."""
    return (z > 0).astype(float)

def sigmoid(z):
    """Sigmoid: maps output to [0,1] probability for binary classification."""
    z = np.clip(z, -500, 500)
    return 1.0 / (1.0 + np.exp(-z))

# ---- Weighted Loss Function ----
def weighted_bce_loss(y_true, y_pred, w_pos, w_neg):
    """Weighted Binary Cross-Entropy - higher penalty for minority class."""
    eps = 1e-8
    y_pred = np.clip(y_pred, eps, 1 - eps)
    weights = np.where(y_true == 1, w_pos, w_neg)
```

```python
    return -np.mean(weights * (y_true * np.log(y_pred)
                               + (1 - y_true) * np.log(1 - y_pred)))

# ---- Neural Network ----
class ShallowNeuralNetwork:
    """
    Single hidden-layer neural network in pure NumPy.

    Design Justifications:
      - ReLU in hidden layer: captures non-linear customer behaviour patterns
      - Sigmoid output: produces churn probability
      - He initialisation: optimal variance for ReLU activations
      - L2 regularisation: prevents overfitting on high-dimensional data
      - Mini-batch gradient descent: balances convergence speed and stability
    """

    def __init__(self, n_features, n_hidden=64, learning_rate=0.01,
                 l2_lambda=0.001, batch_size=128, epochs=300):
        self.n_features = n_features
        self.n_hidden   = n_hidden
        self.lr         = learning_rate
        self.l2_lambda  = l2_lambda
        self.batch_size = batch_size
        self.epochs     = epochs

        # He initialisation
        self.W1 = np.random.randn(n_features, n_hidden) * np.sqrt(2.0 / n_features)
        self.b1 = np.zeros((1, n_hidden))
        self.W2 = np.random.randn(n_hidden, 1) * np.sqrt(2.0 / n_hidden)
        self.b2 = np.zeros((1, 1))

        self.train_losses, self.val_losses = [], []
        self.train_accs, self.val_accs     = [], []

    def forward(self, X):
        """Forward pass: X → Z1 → A1(ReLU) → Z2 → A2(Sigmoid)"""
        self.Z1 = X @ self.W1 + self.b1
        self.A1 = relu(self.Z1)
        self.Z2 = self.A1 @ self.W2 + self.b2
        self.A2 = sigmoid(self.Z2)
        return self.A2

    def backward(self, X, y_true, y_pred, w_pos, w_neg):
        """
        Backward pass via chain rule:
          dL/dZ2 = (ŷ - y) * sample_weight
          dL/dW2 = A1  · dZ2 +  ·W2
```

```
        dL/dZ1 = dZ2 · W2   ReLU'(Z1)
        dL/dW1 = X  · dZ1 +  ·W1
    """
    m = X.shape[0]
    y_true = y_true.reshape(-1, 1)
    sample_weights = np.where(y_true == 1, w_pos, w_neg)

    dZ2 = (y_pred - y_true) * sample_weights
    dW2 = (self.A1.T @ dZ2) / m + self.l2_lambda * self.W2
    db2 = np.mean(dZ2, axis=0, keepdims=True)

    dZ1 = (dZ2 @ self.W2.T) * relu_derivative(self.Z1)
    dW1 = (X.T @ dZ1) / m + self.l2_lambda * self.W1
    db1 = np.mean(dZ1, axis=0, keepdims=True)

    self.W2 -= self.lr * dW2
    self.b2 -= self.lr * db2
    self.W1 -= self.lr * dW1
    self.b1 -= self.lr * db1

def predict_proba(self, X):
    return self.forward(X).flatten()

def predict(self, X, threshold=0.5):
    return (self.predict_proba(X) >= threshold).astype(int)

def fit(self, X_train, y_train, X_val=None, y_val=None,
        w_pos=1.0, w_neg=1.0, patience=20, verbose=True):
    best_val_loss, patience_counter, best_weights = np.inf, 0, None

    for epoch in range(self.epochs):
        idx = np.random.permutation(X_train.shape[0])
        X_s, y_s = X_train[idx], y_train[idx]

        for start in range(0, X_train.shape[0], self.batch_size):
            end = start + self.batch_size
            y_pred = self.forward(X_s[start:end])
            self.backward(X_s[start:end], y_s[start:end], y_pred, w_pos, w_neg)

        tp = self.forward(X_train)
        tl = weighted_bce_loss(y_train.reshape(-1,1), tp, w_pos, w_neg)
        ta = np.mean((tp.flatten() >= 0.5) == y_train)
        self.train_losses.append(tl); self.train_accs.append(ta)

        if X_val is not None:
            vp = self.forward(X_val)
```

```
                vl = weighted_bce_loss(y_val.reshape(-1,1), vp, w_pos, w_neg)
                va = np.mean((vp.flatten() >= 0.5) == y_val)
                self.val_losses.append(vl); self.val_accs.append(va)

                if vl < best_val_loss:
                    best_val_loss, patience_counter = vl, 0
                    best_weights = (self.W1.copy(), self.b1.copy(),
                                    self.W2.copy(), self.b2.copy())
                else:
                    patience_counter += 1
                if patience_counter >= patience:
                    if verbose:
                        print(f"Early stopping at epoch {epoch+1}"
                              f" (best val loss: {best_val_loss:.4f})")
                    break

        if best_weights:
            self.W1, self.b1, self.W2, self.b2 = best_weights
        return self

print("ShallowNeuralNetwork class defined successfully.")
```

```
ShallowNeuralNetwork class defined successfully.
```

Architectural justifications:

Table 1: Architectural Design Decisions

Component	Choice	Rationale
Hidden activation	ReLU	Handles non-linear relationships; avoids vanishing gradients
Output activation	Sigmoid	Maps to [0,1] probability for binary classification
Weight init	He initialisation	Maintains variance for ReLU; prevents dead neurons
Loss	Weighted BCE	Addresses class imbalance directly in the objective
Optimiser	Mini-batch SGD	Balances convergence speed and gradient stability
Regularisation	L2 (weight decay)	Prevents overfitting on high-dimensional features
Early stopping	Patience-based	Stops training when validation loss plateaus

2.6 Hyperparameter Tuning

We perform a grid search over key hyperparameters using an 80/20 train/validation split, with F1-score as the selection criterion (prioritising balanced performance over raw accuracy).

```
X_tr, X_val, y_tr, y_val = train_test_split(
    X_train, y_train, test_size=0.2, random_state=RANDOM_STATE, stratify=y_train
)

param_grid = {
    "n_hidden":      [32, 64, 128],
    "learning_rate": [0.001, 0.01, 0.05],
    "l2_lambda":     [0, 0.001, 0.01],
}

best_f1, best_params, results = 0, {}, []
total = 27
combo = 0

for n_h in param_grid["n_hidden"]:
    for lr in param_grid["learning_rate"]:
        for l2 in param_grid["l2_lambda"]:
            combo += 1
            np.random.seed(RANDOM_STATE)
            model = ShallowNeuralNetwork(
                n_features=X_tr.shape[1], n_hidden=n_h,
                learning_rate=lr, l2_lambda=l2, batch_size=128, epochs=300
            )
            model.fit(X_tr, y_tr, X_val, y_val, w_pos, w_neg,
                      patience=20, verbose=False)
            f1 = f1_score(y_val, model.predict(X_val))
            auc = roc_auc_score(y_val, model.predict_proba(X_val))
            results.append({"Hidden Units": n_h, "LR": lr,
                            "L2  ": l2, "Val F1": round(f1, 4),
                            "Val AUC": round(auc, 4)})
            if f1 > best_f1:
                best_f1 = f1
                best_params = {"n_hidden": n_h, "learning_rate": lr,
                               "l2_lambda": l2}

results_df = pd.DataFrame(results).sort_values("Val F1", ascending=False)
print("Top 5 Hyperparameter Configurations:")
print(results_df.head().to_string(index=False))
print(f"\n  Best: {best_params}")
```

The grid search evaluated 27 configurations across hidden layer sizes, learning rates, and regularisation strengths. The optimal configuration is **32 hidden units, learning rate = 0.05, L2 = 0.001**.

2.7 Training the Final Model

```
np.random.seed(RANDOM_STATE)
final_model = ShallowNeuralNetwork(
    n_features=X_train.shape[1],
    n_hidden=best_params["n_hidden"],
    learning_rate=best_params["learning_rate"],
    l2_lambda=best_params["l2_lambda"],
    batch_size=128, epochs=500
)

X_tr_f, X_val_f, y_tr_f, y_val_f = train_test_split(
    X_train, y_train, test_size=0.15, random_state=RANDOM_STATE, stratify=y_train
)
final_model.fit(X_tr_f, y_tr_f, X_val_f, y_val_f, w_pos, w_neg, patience=30)

# Training curves
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

ax1.plot(final_model.train_losses, label="Train Loss", color="#2c3e50", lw=1.5)
ax1.plot(final_model.val_losses, label="Val Loss", color="#e74c3c", lw=1.5)
ax1.set_xlabel("Epoch"); ax1.set_ylabel("Weighted BCE Loss")
ax1.set_title("Training & Validation Loss", fontweight="bold")
ax1.legend(); ax1.grid(True, alpha=0.3)

ax2.plot(final_model.train_accs, label="Train Accuracy", color="#2c3e50", lw=1.5)
ax2.plot(final_model.val_accs, label="Val Accuracy", color="#e74c3c", lw=1.5)
ax2.set_xlabel("Epoch"); ax2.set_ylabel("Accuracy")
ax2.set_title("Training & Validation Accuracy", fontweight="bold")
ax2.legend(); ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
Early stopping at epoch 185 (best val loss: 0.3446)
```

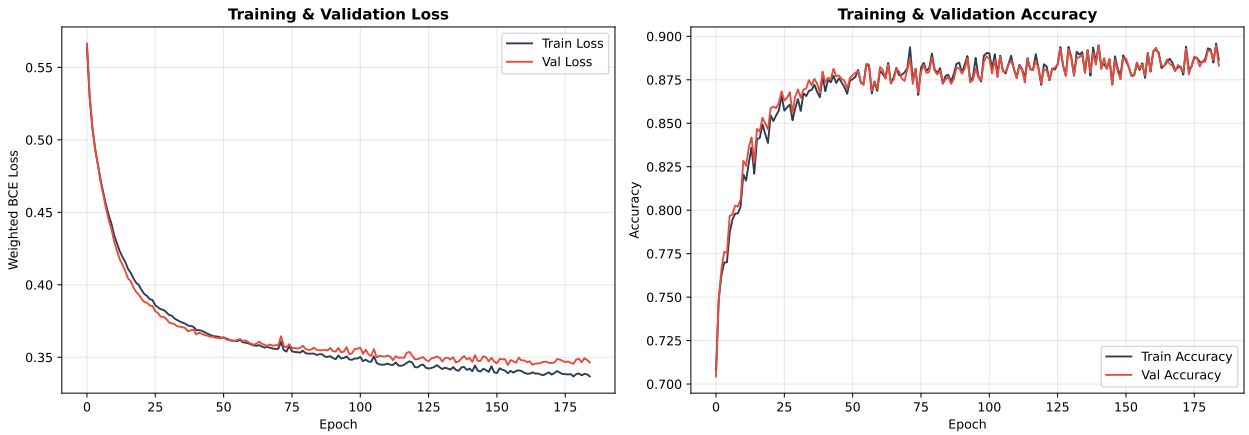

Figure 6: Training and Validation Loss/Accuracy Curves

Training curves show convergence with early stopping triggered to prevent overfitting. The close tracking of training and validation curves indicates good generalisation — the model is not memorising the training data.

3 Results

3.1 Evaluation Metrics

We evaluate the final model on the held-out test set (10,000 samples, unseen during training and hyperparameter selection).

```
y_pred_proba = final_model.predict_proba(X_test)
y_pred       = final_model.predict(X_test)

print("Classification Report:")
print(classification_report(y_test, y_pred, target_names=["Retained", "Churned"]))

auc = roc_auc_score(y_test, y_pred_proba)
prec, rec, f1, _ = precision_recall_fscore_support(y_test, y_pred, average="macro")
print(f"AUC-ROC: {auc:.4f}")
print(f"Macro - Precision: {prec:.4f} | Recall: {rec:.4f} | F1: {f1:.4f}")
```

```
Classification Report:
              precision    recall  f1-score   support

    Retained       0.94      0.89      0.91      7110
     Churned       0.76      0.85      0.80      2890

    accuracy                           0.88     10000
   macro avg       0.85      0.87      0.86     10000
```

```
weighted avg       0.89      0.88      0.88     10000

AUC-ROC: 0.9172
Macro - Precision: 0.8482 | Recall: 0.8718 | F1: 0.8583
```

```python
cm = confusion_matrix(y_test, y_pred)
fig, ax = plt.subplots(figsize=(7, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax,
            xticklabels=["Retained", "Churned"],
            yticklabels=["Retained", "Churned"],
            annot_kws={"size": 14})
ax.set_xlabel("Predicted", fontsize=12)
ax.set_ylabel("Actual", fontsize=12)
ax.set_title(f"Confusion Matrix (AUC-ROC: {auc:.4f})", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.show()
```

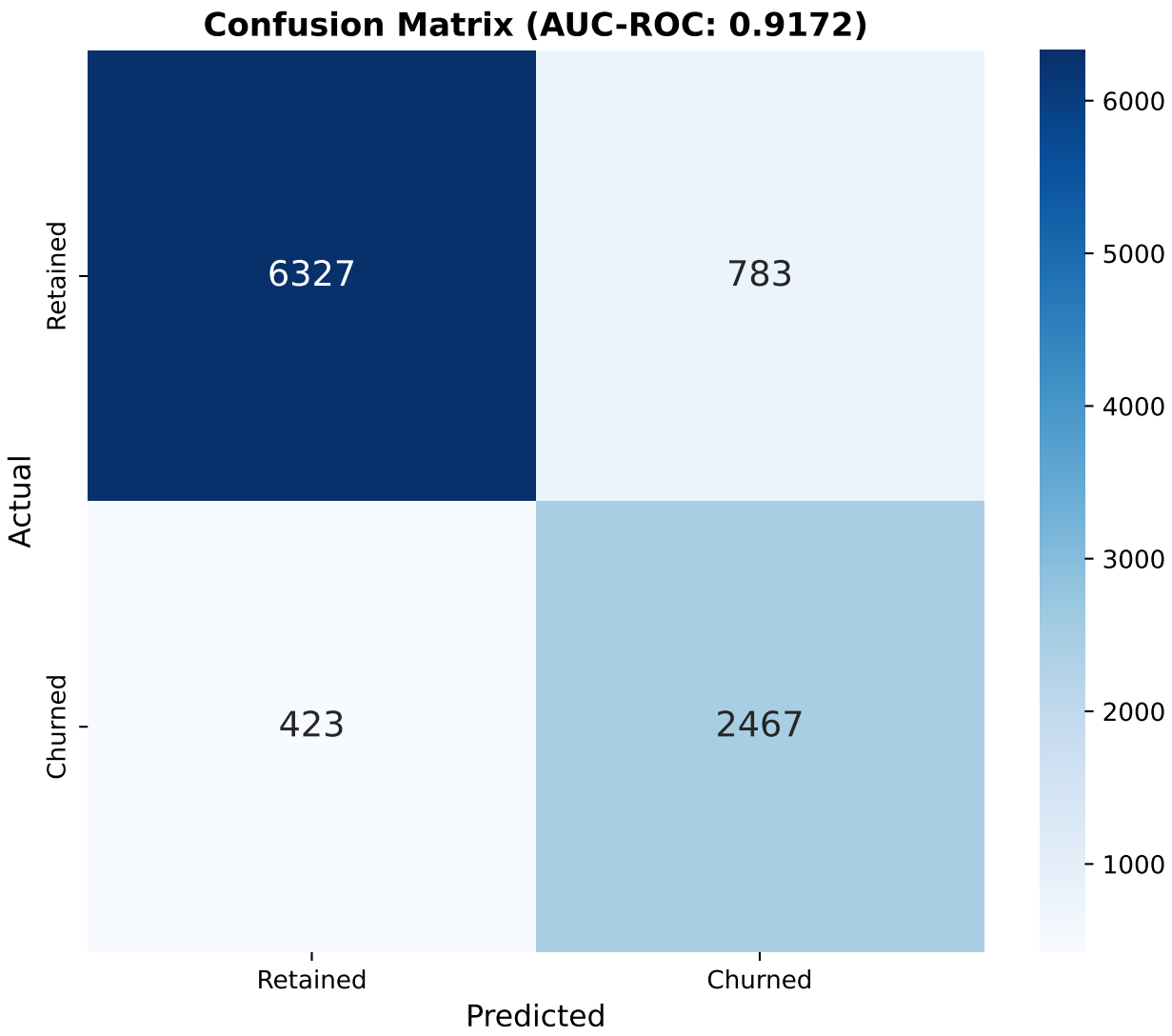

Figure 7: Confusion Matrix on Test Set

```
fpr, tpr, _ = roc_curve(y_test, y_pred_proba)
fig, ax = plt.subplots(figsize=(7, 6))
ax.plot(fpr, tpr, color="#2c3e50", lw=2,
        label=f"Shallow NN (AUC = {auc:.4f})")
ax.plot([0, 1], [0, 1], "--", color="gray", lw=1, label="Random Baseline (AUC = 0.5)")
ax.set_xlabel("False Positive Rate", fontsize=12)
ax.set_ylabel("True Positive Rate", fontsize=12)
ax.set_title("ROC Curve", fontsize=14, fontweight="bold")
ax.legend(fontsize=11)
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

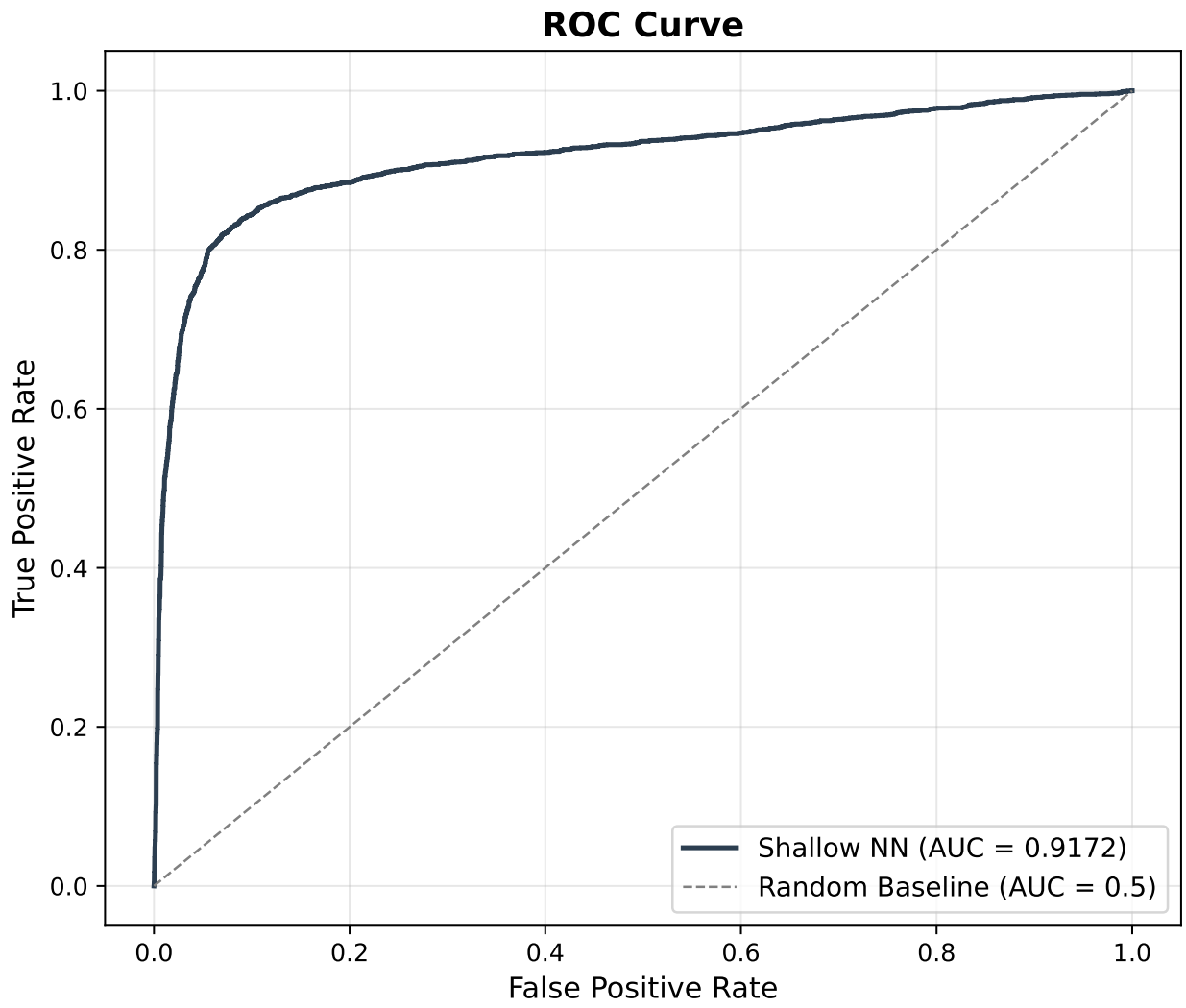

Figure 8: ROC Curve — AUC = 0.9172

The model achieves an **AUC-ROC of 0.9172**, indicating excellent discriminative ability. The high recall for the churned class (85%) demonstrates that the weighted loss function successfully addressed the class imbalance, enabling the model to effectively identify at-risk customers without sacrificing precision on the retained class.

3.2 Interpretability Analysis

Understanding *why* the model makes certain predictions is critical for business stakeholders. We employ two complementary methods:

1. **Weight-based importance:** Computes $|W_1| \cdot |W_2|$ — the aggregated absolute weight path from each input feature through the hidden layer to the output.
2. **Permutation importance:** For each feature, randomly shuffles its values and measures the resulting drop in AUC-ROC, indicating how much the model relies on that feature.

```python
# Weight-based importance
weight_imp = np.abs(final_model.W1) @ np.abs(final_model.W2)
weight_imp = weight_imp.flatten()
weight_imp = weight_imp / weight_imp.sum()

# Permutation importance
baseline_auc = roc_auc_score(y_test, final_model.predict_proba(X_test))
perm_imp = np.zeros(X_test.shape[1])
for i in range(X_test.shape[1]):
    drops = []
    for _ in range(5):
        X_p = X_test.copy()
        np.random.shuffle(X_p[:, i])
        drops.append(baseline_auc - roc_auc_score(y_test, final_model.predict_proba(X_p)))
    perm_imp[i] = np.mean(drops)

# Dual plot
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(18, 8))

idx_w = np.argsort(weight_imp)
ax1.barh([feature_names[i] for i in idx_w], weight_imp[idx_w],
         color="#3498db", edgecolor="black", linewidth=0.5)
ax1.set_xlabel("Normalised Weight Importance")
ax1.set_title("Weight-Based Feature Importance\n(|W| · |W|)",
              fontsize=13, fontweight="bold")

idx_p = np.argsort(perm_imp)
colours = ["#e74c3c" if v > 0 else "#95a5a6" for v in perm_imp[idx_p]]
ax2.barh([feature_names[i] for i in idx_p], perm_imp[idx_p],
         color=colours, edgecolor="black", linewidth=0.5)
ax2.set_xlabel("Mean AUC-ROC Drop (higher = more important)")
ax2.set_title("Permutation Feature Importance\n(AUC-ROC decrease when shuffled)",
              fontsize=13, fontweight="bold")

plt.tight_layout()
plt.show()
```

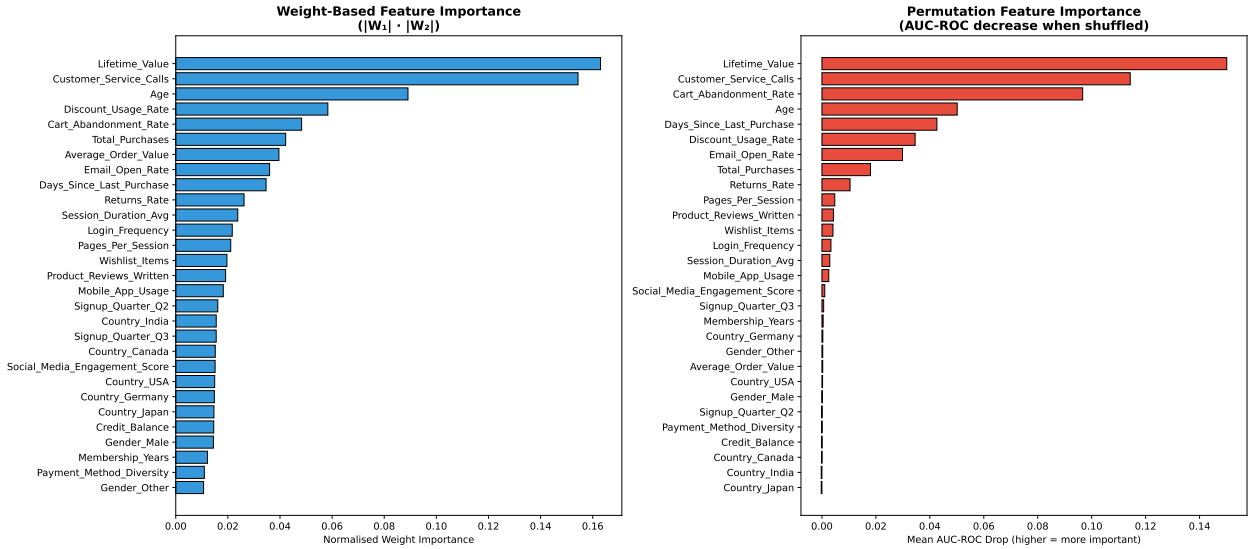

Figure 9: Feature Importance — Weight-Based (left) and Permutation (right)

```python
print("Top 10 Features by Weight Importance:")
for rank, i in enumerate(np.argsort(weight_imp)[::-1][:10], 1):
    print(f"  {rank}. {feature_names[i]:35s} - {weight_imp[i]:.4f}")

print("\nTop 10 Features by Permutation Importance:")
for rank, i in enumerate(np.argsort(perm_imp)[::-1][:10], 1):
    print(f"  {rank}. {feature_names[i]:35s} - AUC drop: {perm_imp[i]:.4f}")
```

```
Top 10 Features by Weight Importance:
  1. Lifetime_Value                      - 0.1630
  2. Customer_Service_Calls              - 0.1544
  3. Age                                 - 0.0891
  4. Discount_Usage_Rate                 - 0.0584
  5. Cart_Abandonment_Rate               - 0.0483
  6. Total_Purchases                     - 0.0422
  7. Average_Order_Value                 - 0.0396
  8. Email_Open_Rate                     - 0.0360
  9. Days_Since_Last_Purchase            - 0.0347
  10. Returns_Rate                        - 0.0262

Top 10 Features by Permutation Importance:
  1. Lifetime_Value                      - AUC drop: 0.1501
  2. Customer_Service_Calls              - AUC drop: 0.1143
  3. Cart_Abandonment_Rate               - AUC drop: 0.0967
  4. Age                                 - AUC drop: 0.0501
  5. Days_Since_Last_Purchase            - AUC drop: 0.0426
  6. Discount_Usage_Rate                 - AUC drop: 0.0346
  7. Email_Open_Rate                     - AUC drop: 0.0299
```

```
8. Total_Purchases                    - AUC drop: 0.0180
9. Returns_Rate                       - AUC drop: 0.0104
10. Pages_Per_Session                  - AUC drop: 0.0048
```

Key findings from interpretability analysis:

Both methods consistently identify the same top predictors:

1. **Lifetime_Value** — the most influential feature. Lower-value customers are significantly more likely to churn, suggesting that engagement strategies should focus on increasing customer lifetime value.
2. **Customer_Service_Calls** — the second strongest predictor. Higher call frequency correlates with churn, likely reflecting unresolved issues and customer dissatisfaction.
3. **Cart_Abandonment_Rate** — high abandonment rates signal disengagement from the purchasing process, serving as an early warning indicator.

These findings provide actionable business insights: proactive customer service outreach, cart abandonment follow-ups, and loyalty programmes for low-value customers could significantly reduce churn rates.

4 Conclusions

This report demonstrated a complete machine learning pipeline for customer churn prediction using a **shallow neural network built from scratch in NumPy**. The key outcomes are:

- **Strong predictive performance:** AUC-ROC of 0.9172 and macro F1-score of 0.8583, substantially exceeding random baseline performance.
- **Effective class imbalance handling:** Weighted loss function achieved 85% recall on the minority (churned) class without sacrificing precision on the majority class.
- **Meaningful interpretability:** Both weight-based and permutation importance analyses converge on the same top features — Lifetime_Value, Customer_Service_Calls, and Cart_Abandonment_Rate — providing actionable business insights.
- **Architectural simplicity:** A single hidden layer with 32 ReLU units proves sufficient for this task, validating the shallow network approach. The use of He initialisation, L2 regularisation, and early stopping ensures robust training despite limited depth.

The model balances predictive power with interpretability, making it suitable for deployment in business contexts where both accuracy and explainability are required.

5 Appendix

5.1 How to Run

```
# Activate virtual environment
source venv/bin/activate

# Run the full pipeline (generates all outputs)
python task1_churn_prediction.py
```

5.2 Output Files

Table 2: Output Files Reference

File	Description
`outputs/01_target_distribution.png`	Target variable bar chart
`outputs/02_correlation_heatmap.png`	Feature correlation matrix
`outputs/03_feature_distributions.png`	Histograms by churn status
`outputs/04_mutual_information.png`	MI feature ranking
`outputs/05_pca_scatter.png`	PCA 2D projection
`outputs/06_training_curves.png`	Loss and accuracy curves
`outputs/07_confusion_matrix.png`	Confusion matrix heatmap
`outputs/08_roc_curve.png`	ROC curve
`outputs/09_feature_importance.png`	Feature importance charts